

CodeRisk Agent

AI-Powered Code Security Analysis · Running Entirely on Local AMD GPU

Track 2: Agentic AI

Team: Yang Weike (Captain) + Iolo (AI Assistant)

Repository: github.com/a9320/code-risk-agent

Date: July 2026

🔍 The Enterprise Code Security Dilemma

Traditional Static Analysis (Semgrep, Clang-Tidy)

- ▶ ✗ Pure pattern matching — misses logic bugs
- ▶ ✗ No reasoning about code intent
- ▶ ✗ High false positive rate

Cloud AI (GitHub Copilot, Snyk)

- ▶ ✗ Source code uploaded to cloud
- ▶ ✗ HIPAA / GDPR / SOX compliance risk
- ▶ ✗ Black-box analysis — no evidence chain

Enterprises need **AI depth** but cannot sacrifice **code security**

CodeRisk Agent = **AI Depth** + **Local Security**

Understands code logic (like cloud AI) · Runs on local AMD GPU (like static analysis)

💡 CodeRisk Agent — One Command, 4 Agents



Orchestrator State Machine



Agent 1

Static Analyzer



Agent 2

Semantic Analyzer



Agent 3

Deep Verifier



Agent 4

Report Generator

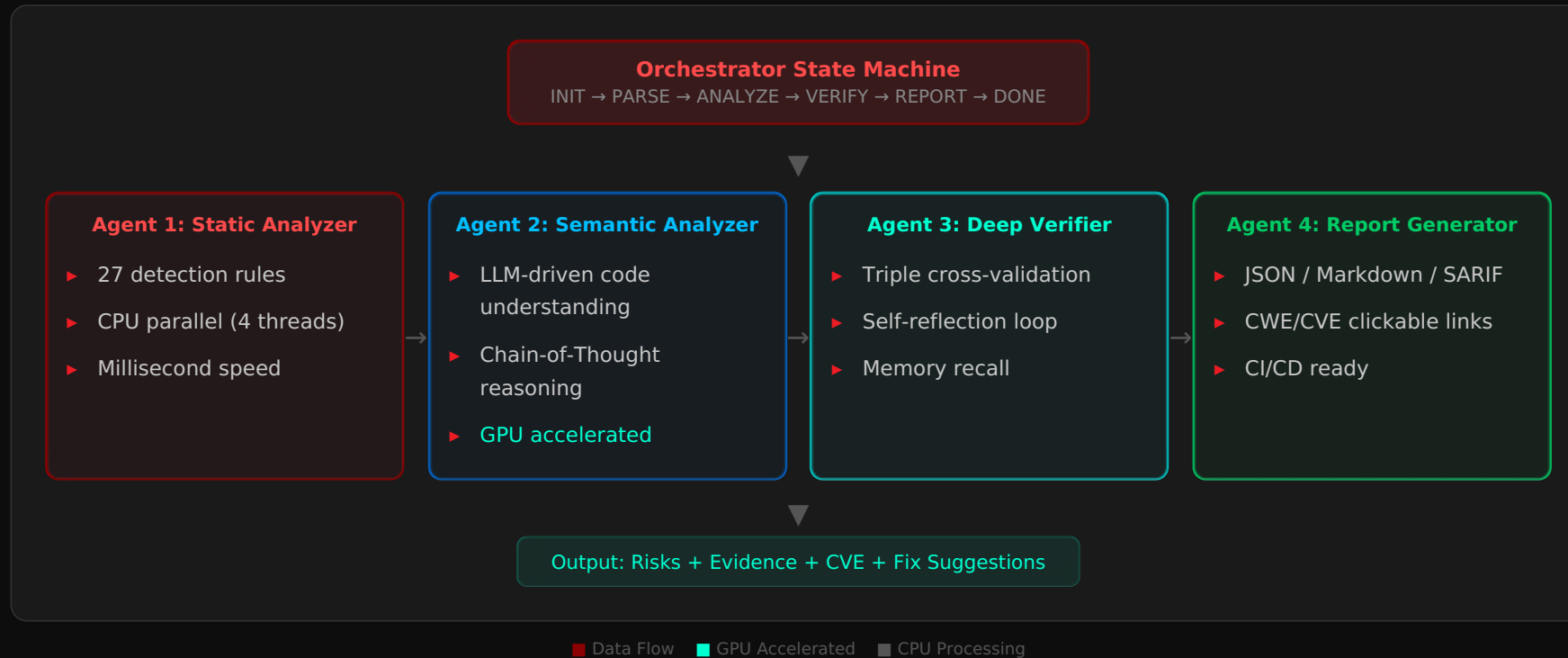
Output

- ▶ Risks + CWE/CVE
- ▶ Attack scenarios
- ▶ Fix suggestions
- ▶ Evidence chains

Key Properties

- ▶ ✓ 100% local execution
- ▶ ✓ Zero cloud API calls
- ▶ ✓ Code never leaves machine

System Architecture





4 Specialized Agents

1 Static Analyzer

🔍 Pattern Detection

- ▶ 27 rules (C: 13, Python: 14)
- ▶ Dangerous functions
- ▶ Vulnerability patterns
- ▶ CPU parallel
- ▶ Millisecond speed

2 Semantic Analyzer

🧠 LLM Understanding

- ▶ LLM-driven analysis
- ▶ CoT structured reasoning
- ▶ Validates Agent 1
- ▶ Discovers missed vulns
- ▶ GPU inference

3 Deep Verifier

⚙️ What Makes It Different

🔑 Triple Cross-Validation

- ▶ Tool confirmation (Semgrep + taint)
- ▶ CWE knowledge base (MITRE ATT&CK)
- ▶ Local CVE database (69,686 entries)

→ **Reduces false positives**

🔄 Self-Reflection Loop

Round 1: Validate existing findings

Round 2: **"What did we miss?"**

Discovers **4 additional risks** per scan

→ **Higher detection rate**

🧠 Dual Memory System

Correct Memory: Confirmed TPs → boost confidence

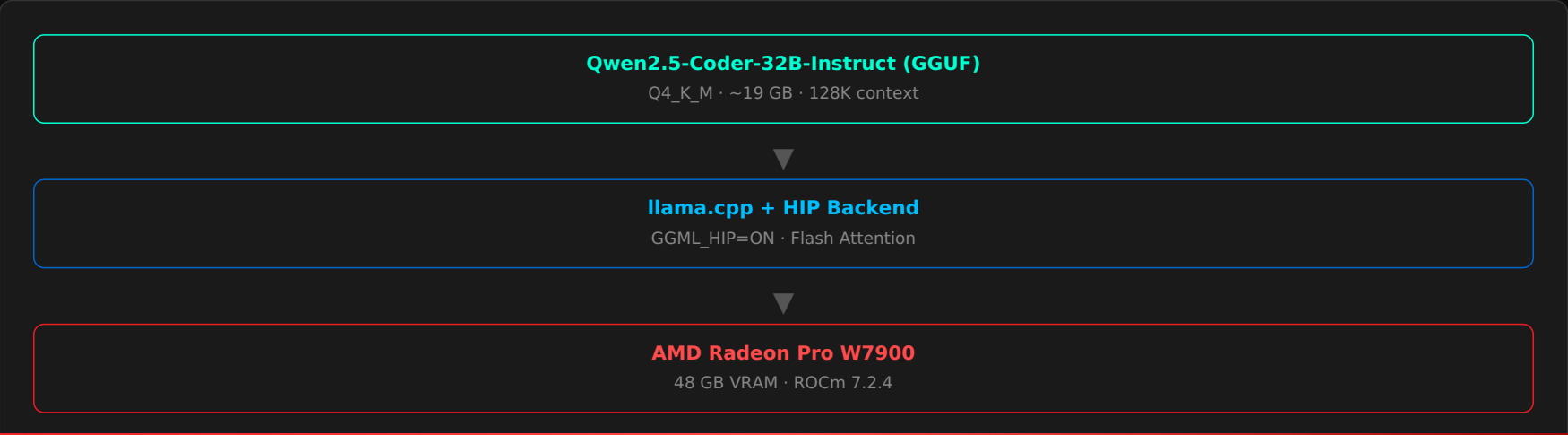
Error Memory: Confirmed FPs → suppress noise

⚡ AMD ROCm GPU Optimization

Optimization	Config	Effect
HIP Backend	<code>GGML_HIP=ON</code>	AMD GPU acceleration
Layer Offload	<code>-ngl 999</code>	All layers on GPU
Flash Attention	<code>-fa 1</code>	30-50% latency reduction
4-bit Quantization	Q4_K_M (GGUF)	19.6 GB / 48 GB VRAM

Key Design Decision

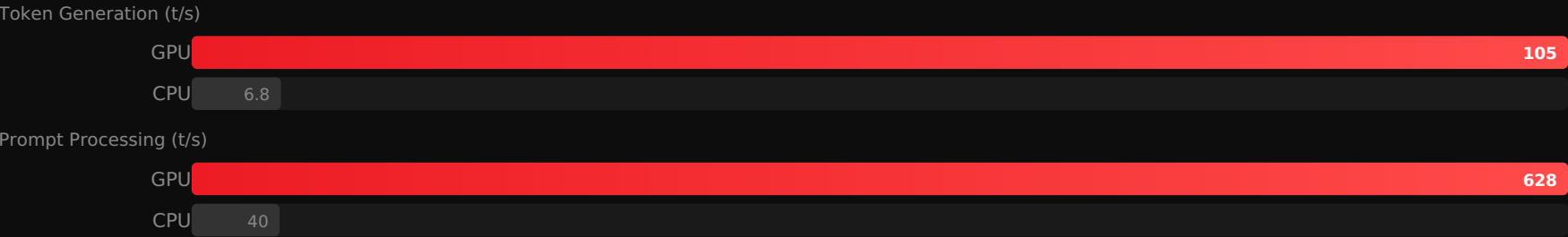
Separate llama-server process via HTTP API
→ Independent scaling · Model swapping · Process isolation



Performance Benchmarks

AMD Radeon Pro W7900 (48 GB) · ROCm 7.2.4 · Qwen2.5-Coder-32B-Instruct Q4_K_M

LLM Inference Speed



Speedup: 15.4x

End-to-End Analysis

Metric	Value
Test Files	5 (C + Python)
Risks Detected	47-48
Total Time	~8 min (32B, ROCm 7.2.4)
CPU Equivalent	>2 hours

Agent Timing

Agent	Time	%
Agent 1: Static	0.004 s	0.0%

Analysis Output — Real Results

Terminal Output

```
CodeRisk Agent - Analysis Report

Files analyzed: 5
Total risks: 47
Analysis time: ~8 min (32B)

Severity | CWE | Title |
CRITICAL | CWE-120 | Buffer overflow |
CRITICAL | CWE-78 | Command injection |
HIGH | CWE-415 | Double free |
HIGH | CWE-502 | Deserialization |
MEDIUM | CWE-476 | NULL dereference |
```

Risk Detail

RISK-014

Buffer Overflow via strcpy()

Severity

HIGH

CWE

MEDIUM

9 / 12



 27 Detection Rules · 51 Tests

Detection Rules

Language	Rules	CWE Coverage
C Dangerous Functions	8	CWE-120, 134, 78
C Vulnerability Patterns	5	CWE-476, 415, 22, 327, 190
Python Dangerous Calls	14	CWE-95, 502, 611, 377, 328, 798
Total	27	

Test Coverage

Test Suite	Tests
test_static_analyzer.py	13
test_llm_client.py	15
test_schemas.py	10
test_memory.py	7
test_cve_client.py	6
Total	51

69,686

CVE entries (local SQLite)

10 / 12

🔒 Compliance & Security — Zero External Calls

Runtime Data Sources

Component	Source	Type
LLM Inference	Local GGUF model	Local file
CVE Data	Local SQLite (NVD)	Local file
OSV Data	Local JSON + fallback	Local file
CWE/ATT&CK	Python dictionaries	Local code
Report URLs	Reference links only	User-clicked

Compliance Ready

- ▶ **HIPAA** — Healthcare data stays on-premise
- ▶ **GDPR** — No cross-border data transfer
- ▶ **SOX** — Full audit trail via evidence chains
- ▶ **SOC 2** — Source code never leaves boundary

Data Preparation

- ▶ One-time NVD bulk download → local SQLite
- ▶ OSV feed → local JSON index
- ▶ No API keys required

CodeRisk Agent

AI Code Security · Local AMD GPU · Zero Cloud Dependency

15.4×

GPU vs CPU Speedup

47-48

Risks per Scan

0

External API Calls

Key Numbers

51 unit tests · 27 rules · 4 agents
69,686 CVE entries · 15 ATT&CK

Links & Team

github.com/a9320/code-risk-agent
Yang Weike (Captain) + lolo